

Ejercicios Heap (Answers)

Ej 1 Min-Heap → {10}
 {5, 10}
 {5, 10, 20}
 {2, 5, 20, 10}
 {2, 5, 20, 10, 8}

Ej 2 Max-Heap → {10}
 {10, 5}
 {20, 5, 10}
 {20, 5, 10, 2}
 {20, 8, 10, 2, 5}

Ej 3 a) [1, 3, 5, 7, 9, 8]

si es valido

b) [2, 1, 3, 4, 5]

no es valido

c) [1, 4, 2, 6, 5]

si es valido

Ej 4 [3, 7, 5, 12, 10, 6]

si es valido

Ej 5 a) Arreglar [10, 5, 20, 7] → [5, 10, 20, 7] → [5, 7, 20, 10]

b) Arreglar 1 + [2, 5, 3, 10, 7] → [2, 5, 3, 10, 7, 1] → [2, 5, 1, 10, 7, 3] → [1, 5, 2, 10, 7, 3]

Ej 6 [9, 4, 7, 1, 3, 6, 2] → [9, 4, 2, 1, 3, 6, 7] → [9, 1, 2, 4, 3, 6, 7] → [1, 9, 2, 4, 3, 6, 7] → [1, 3, 2, 4, 9, 6, 7]

Ej 7 no hay error

Ej 8

a) ¿Por qué un heap no es bueno para buscar?

Porque un heap solo garantiza que el padre sea menor que sus hijos en min-heap, o mayor que sus hijos en max-heap.

Pero no tiene orden completo como un BST.

Ejemplo:

Si buscas un número, no puedes decidir fácilmente "voy izquierda" o "voy derecha".

Respuesta corta

Porque el heap no está totalmente ordenado; solo sabemos la relación entre padre e hijos

b) ¿Por qué la inserción en el heap es $O(\log n)$?

Porque insertas el nuevo elemento al final y luego puede subir por el árbol usando heapify up.

La altura de un heap completo es aproximadamente:

$\log n$

Entonces, en el peor caso, el elemento sube desde una hoja hasta la raíz.

Respuesta corta

Porque el nuevo elemento solo puede subir por la altura del árbol, y la altura es $O(\log n)$

c) ¿Por qué un heap debe ser un árbol binario completo?

Porque así se puede representar fácilmente con un array, sin espacios vacíos raros en medio.

Además, mantenerlo completo asegura que la altura sea pequeña.

Respuesta corta

Porque así se mantiene balanceado y puede guardarse eficientemente en un array.